

Algoritmi e Strutture Dati – 12/01/2026

Esercizio A1 – Complessità – Punti ≥ 8

Sia $n = \text{end} - \text{start} + 1$ la dimensione del sottovettore considerato.

Nel caso base ($n = 1$) l'algoritmo restituisce $A[\text{start}]$ in tempo costante, quindi $T(1) = \Theta(1)$.

Nel caso ricorsivo, oltre alle due chiamate ricorsive su metà intervallo, c'è un ciclo che dimezza la dimensione del vettore a ogni passo: $i = \lfloor i/2 \rfloor$. Il numero di iterazioni è quindi $\Theta(\log n)$, e ciascuna iterazione fa lavoro $O(1)$, quindi il costo del ciclo è $\Theta(\log n)$.

La ricorrenza risultante è quindi

$$T(n) = \begin{cases} \Theta(1) & n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(\log n) & n > 1. \end{cases}$$

Per semplificare, possiamo assumere $n = 2^h$ per qualche $h \geq 0$. In questo modo mid divide esattamente l'intervallo in due parti di dimensione $n/2$ e la ricorrenza si scrive:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(\log n).$$

Questa assunzione non cambia l'ordine di grandezza del risultato: per valori generali di n , i due sottoproblemi hanno dimensione $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$, e la profondità dell'albero di ricorsione resta $\Theta(\log n)$.

Confrontiamo la ricorrenza con la forma

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

dove qui $a = 2$, $b = 2$, e $f(n) = \Theta(\log n)$.

Calcoliamo

$$\alpha = \log_b a = \log_2 2 = 1$$

Il Master Theorem confronta $f(n)$ con n^α , usando un parametro $\epsilon > 0$ per formalizzare il fatto che $f(n)$ sia *polinomialmente più piccola* oppure *più grande*.

Nel nostro caso, osserviamo che per ogni $\epsilon > 0$ vale

$$\log n = O(n^\epsilon).$$

Di conseguenza

$$f(n) = \Theta(\log n) = O(n^\epsilon) = O(n^{1-(1-\epsilon)}) = O(n^{1-\delta})$$

ponendo $\delta = 1 - \epsilon > 0$ sufficientemente piccolo. Equivalentemente, possiamo scrivere direttamente:

$$\log n = O(n^{1-\epsilon}) \quad \text{per ogni } \epsilon \in (0, 1),$$

e quindi

$$f(n) = O(n^{\log_b a - \epsilon}) \quad (\text{con lo stesso } \epsilon \in (0, 1)).$$

Questo è esattamente il **caso 1** del Master Theorem esteso:

$$f(n) = O(n^{\log_b a - \epsilon}) \text{ per qualche } \epsilon > 0 \implies T(n) = \Theta(n^{\log_b a}) = \Theta(n).$$

La complessità computazionale dell'algoritmo è

$$T(n) = \Theta(n).$$

Soluzioni errate . Alcuni studenti non si sono accorti del dimezzamento e hanno proposto un'equazione di ricorrenza con n al posto di $\log n$.

Esercizio A2 – Alberobello – Punti ≥ 10

Questo problema (ispirato a un problema simile del 14/07/2025) richiede di contare tutti i nodi che sono radici di sottoalberi che soddisfano la proprietà di essere alberobelli.

Osserviamo che la proprietà di essere un alberobello è strutturale e può essere verificata risalendo dai livelli più bassi dell'albero. In particolare, un nodo non può essere radice di un alberobello se nel suo sottoalbero compare almeno un nodo con un solo figlio.

Ne segue che un nodo t è radice di un alberobello se e solo se vale una delle seguenti condizioni:

- t è una foglia;
- t ha due figli e i sottoalberi radicati in entrambi i figli sono alberobelli.

La funzione principale richiama una funzione ricorsiva che, dato un nodo t , restituisce due valori:

- il numero totale di radici alberobelle contenute nel sottoalbero radicato in t ;
- un valore booleano che vale **true** se il sottoalbero radicato in t è un alberobello, **false** altrimenti.

La funzione ricorsiva opera come segue:

- Se $t = \text{nil}$, non ci sono radici alberobelle e il sottoalbero non è considerato un alberobello.
- Se t è una foglia, allora il sottoalbero radicato in t è un alberobello e contribuisce con una unità al conteggio.
- In tutti gli altri casi, la funzione richiama ricorsivamente i due figli. Il nodo t è una radice alberobella se e solo se entrambi i sottoalberi figli sono alberobelli; in tal caso il conteggio viene incrementato di uno. Altrimenti, il conteggio è semplicemente la somma dei conteggi dei due sottoalberi.

La correttezza dell'algoritmo segue per induzione strutturale sull'albero, poiché la decisione se un nodo è radice di un alberobello dipende esclusivamente dalle informazioni restituite dai suoi figli.

La complessità computazionale è $O(n)$, in quanto ogni nodo dell'albero viene visitato esattamente una volta durante una visita in post-ordine.

```

int countAB(TREE  $T$ )
    int, boolean  $tot, isNice = \text{abRec}(T)$ 
    return  $tot$ 

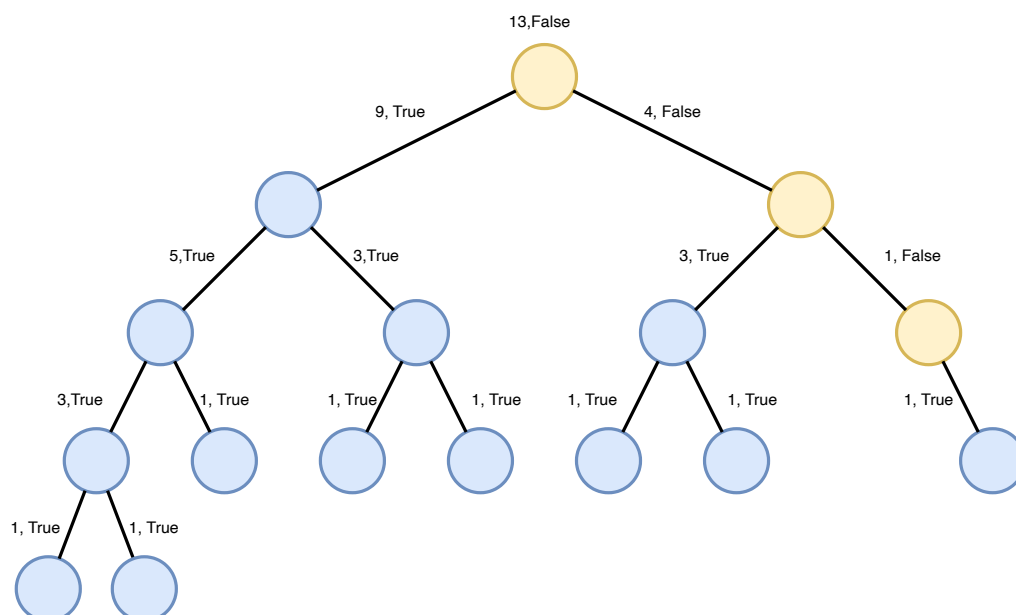
```

```

int, boolean abRec(TREE  $t$ )
    if  $t == \text{nil}$  then
        | return 0, false
    if  $t.\text{left} == t.\text{right} == \text{nil}$  then
        | return 1, true
    int, boolean  $tot_L, isNice_L = \text{abRec}(t.\text{left})$ 
    int, boolean  $tot_R, isNice_R = \text{abRec}(t.\text{right})$ 
    if  $isNice_L$  and  $isNice_R$  then
        | return  $\langle tot_L + tot_R + 1, \text{true} \rangle$ 
    else
        | return  $\langle tot_L + tot_R, \text{false} \rangle$ 

```

La figura seguente mostra i valori ritornati nell'albero di esempio.



Esercizio A3 – Massimo raggiungibile (0-based) – Punti ≥ 10

È possibile risolvere il problema adattando l'algoritmo per il calcolo delle componenti connesse. L'osservazione chiave è che, in un grafo non orientato, due nodi u e v sono reciprocamente raggiungibili se e solo se appartengono alla stessa componente connessa. Quindi, per ogni nodo u , il valore $M[u]$ richiesto coincide con il massimo valore $value$ fra tutti i nodi della componente connessa di u .

L'algoritmo procede in tre fasi:

1. esegue una visita in profondità (DFS) per etichettare ogni nodo con l'indice della propria componente connessa;
2. associa ad ogni componente il valore massimo;
3. costruisce infine il vettore M assegnando a ogni nodo u il valore massimo associato alla sua componente.

```
int[] maxReachable(GRAPH G)
```

```
% Identifica le componenti connesse e memorizza in nComponents il loro numero
int[] id = cc(G)
int nComponents = max(id)

% Calcola il massimo valore per ogni componente, memorizzato in maxComponent
int[] maxComponent = new int[0...nComponents] = {-∞}
foreach u ∈ G.V() do
    | maxComponent[id[u]] = max(maxComponent[id[u]], u.value)

% Crea il vettore di ritorno associando a ogni nodo il valore massimo della sua componente
int[] M = new int[0...G.size() - 1]
foreach u ∈ G.V() do
    | M[u] = maxComponent[id[u]]

return M
```

L'algoritmo inizia chiamando la funzione `cc()`, tramite la quale associamo ad ogni nodo un valore numerico da 1 a $nComponents$, che identifica la componente di appartenenza; in altre parole, due nodi con lo stesso valore appartengono alla stessa componente. Siccome la versione del libro non restituisce il valore $nComponents$, lo calcoliamo con una semplice operazione `max()` sul vettore `id`.

A questo punto, calcoliamo il vettore `maxComponent` in modo che `maxComponent[k]` contenga il valore massimo associato alla componente k . Questo viene fatto scorrendo ogni nodo u e calcolando il massimo fra il più alto valore trovato per la componente di u e $u.value$. Essendo un semplice ciclo sui nodi, questa operazione ha costo $\Theta(n)$.

Infine, per ogni nodo u , ponendo $M[u] = maxComponent[id[u]]$ assegniamo a $M[u]$ il massimo valore della componente connessa di u . Poiché in un grafo non orientato l'insieme dei nodi raggiungibili da u coincide con la componente connessa di u , segue che $M[u]$ è esattamente il massimo valore associato a un nodo raggiungibile da u , come richiesto. Essendo un semplice ciclo sui nodi, questa operazione ha costo $\Theta(n)$.

Complessità. La complessità finale è quindi $\Theta(m + n)$.

Possibili errori e soluzioni alternative

- Alcuni non hanno considerato la possibilità che il grafo non sia connesso. Se il grafo è connesso, tutti i nodi sono raggiungibili fra di loro e quindi non sarebbe nemmeno necessaria una visita, basterebbe restituire un vettore contenente il valore massimo in ogni posizione. Se questa è la soluzione che avete in mente, fatevi due domande: veramente questo è un esercizio da 12 punti? È più facile che vi stiate dimenticando qualcosa.
- Alcuni non hanno letto "non orientato"; c'è poco da dire, il suggerimento è di leggere bene il testo del problema. In questo caso cambia totalmente la natura del problema.
- Un certo numero ha scelto la via "facile" di far partire una visita da ogni nodo, con costo $\Theta(n(m + n)) = \Theta(mn)$. Questo ovviamente non è un errore, ma si può fare meglio di così!
- In molte soluzioni, ho visto semplicemente degli adattamenti di visite in profondità o in ampiezza in due modi:

- Calcolando il valore $M[u]$ in pre-visita, ovvero quando u viene scoperto: cioè memorizzando il valore massimo trovato finora e settando $M[u]$ con il massimo fra u e il massimo valore incontrato finora. Questo ha il problema che non considera valori più alti che possono essere incontrati dopo, quindi l'algoritmo è sbagliato.
- Calcolando il valore $M[u]$ in post-visita, ovvero dopo aver visitato tutti i vicini e prendendo il massimo da tutti i vicini. Questo sembra un approccio migliore, ma ha comunque un problema: u prende il valore massimo fra tutti i discendenti, e questo è corretto; ma non è detto che i discendenti di u prendano il valore massimo di tutta la componente, perché quando vengono visitati non tutti i nodi sono stati scoperti. A titolo di esempio, considerate tre nodi come in figura



Supponendo che l'ordine di scoperta sia $2 - 1 - 3$, il nodo con valore 1 avrà valore massimo 2 e non 3.

Esercizio B1 – Numeri zerabili (0-based) – Punti ≥ 8

Il problema può essere risolto tramite la tecnica del backtrack, con un meccanismo di pruning che interrompe la ricorsione quando le stringhe contengono già un numero eccessivo di cifre 1.

Costruiamo la stringa binaria $S[0 \dots n-1]$ da sinistra a destra con backtracking. La chiamata $\text{zeroRec}(S, n, i, \text{ones})$ ha già fissato le posizioni $0 \dots i-1$ e ones è il numero di cifre 1 nel prefisso costruito. Proviamo sempre a porre $S[i] = 0$ (non aumenta il numero di 1). Proviamo invece $S[i] = 1$ solo se, dopo questa scelta, resta ancora possibile avere più 0 che 1 nell'intera stringa: ciò è garantito dalla condizione

$$\text{ones} + 1 < n - (\text{ones} + 1),$$

che equivale a richiedere che il numero totale di 1 rimanga strettamente minore del numero totale di 0. (Osservazione: la condizione può essere semplificata in $2 \cdot (\text{ones} + 1) < n$.)

Per induzione sulle chiamate ricorsive si mantiene l'invariante che il valore corrente di ones può ancora portare, completando le posizioni rimanenti, a una stringa con più 0 che 1. In particolare, ogni ramo che aggiunge un 1 rispetta la condizione sopra e quindi non può condurre a una stringa non zerabile; di conseguenza, quando $i = n$ la stringa stampata è zerabile. Viceversa, ogni stringa zerabile può essere seguita nelle scelte bit per bit senza mai violare la condizione, quindi viene generata e stampata.

```
printZerable(int n)
```

```
int[] S = new int[0...n-1]
zeroRec(S, n, 0, 0)
```

```
zeroRec(int [] S, int n, int i, int ones)
```

```
if i == n then
  | print S[0...n-1]
else
  | S[i] = 0
  | zeroRec(S, n, i+1, ones)
  | if ones + 1 < n - (ones + 1) then
  |   | S[i] = 1
  |   | zeroRec(S, n, i+1, ones + 1)
```

Un algoritmo che generasse tutte le stringhe binarie di lunghezza n e le stampasse avrebbe costo $\Theta(n2^n)$, poiché vengono prodotte 2^n stringhe e la stampa di ciascuna richiede tempo $\Theta(n)$.

Nel nostro caso, grazie al pruning, vengono generate e stampate solo le stringhe zerabili, il cui numero è

$$\sum_{k=0}^{\lfloor (n-1)/2 \rfloor} \binom{n}{k},$$

cioè tutte le stringhe con un numero di 1 strettamente minore di $n/2$. Questo valore è esponenziale in n , ma circa la metà di 2^n (più precisamente, è asintoticamente $\Theta(2^{n-1})$).

Infatti, il numero di stringhe zerabili di lunghezza $n = 1 \dots 10$ è

n	1	2	3	4	5	6	7	8	9	10
stringhe zerabili#	1	1	4	5	16	22	64	93	256	386

Questa è la sequenza <https://oeis.org/A058622> dell'Online Encyclopedia of Integer Sequences.